

Javascript – DOM

ISC

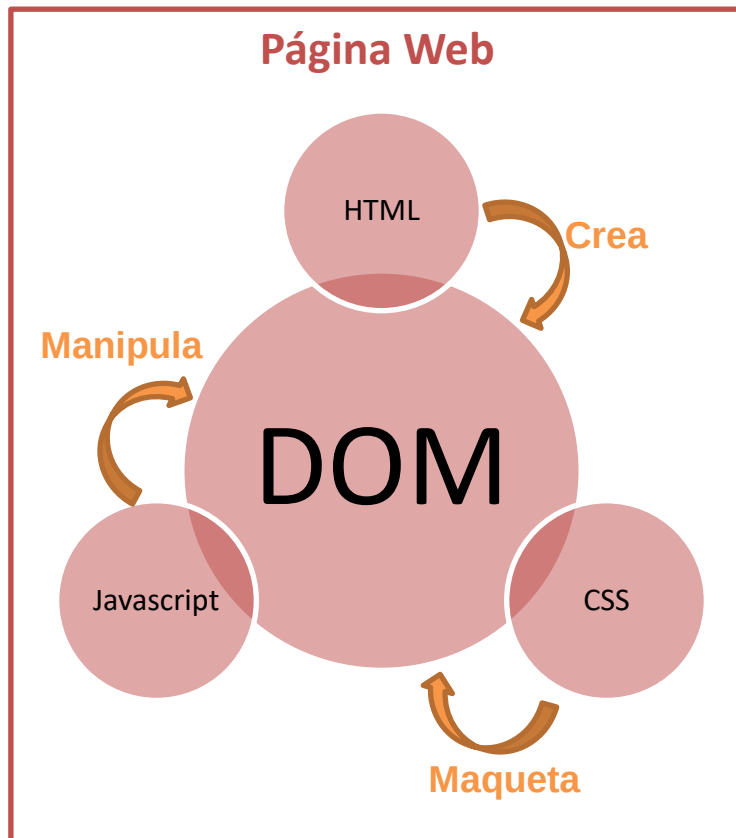
Infraestructura de
Sistemas Clínicos



Universitat d'Alacant
Universidad de Alicante



Esquema DOM



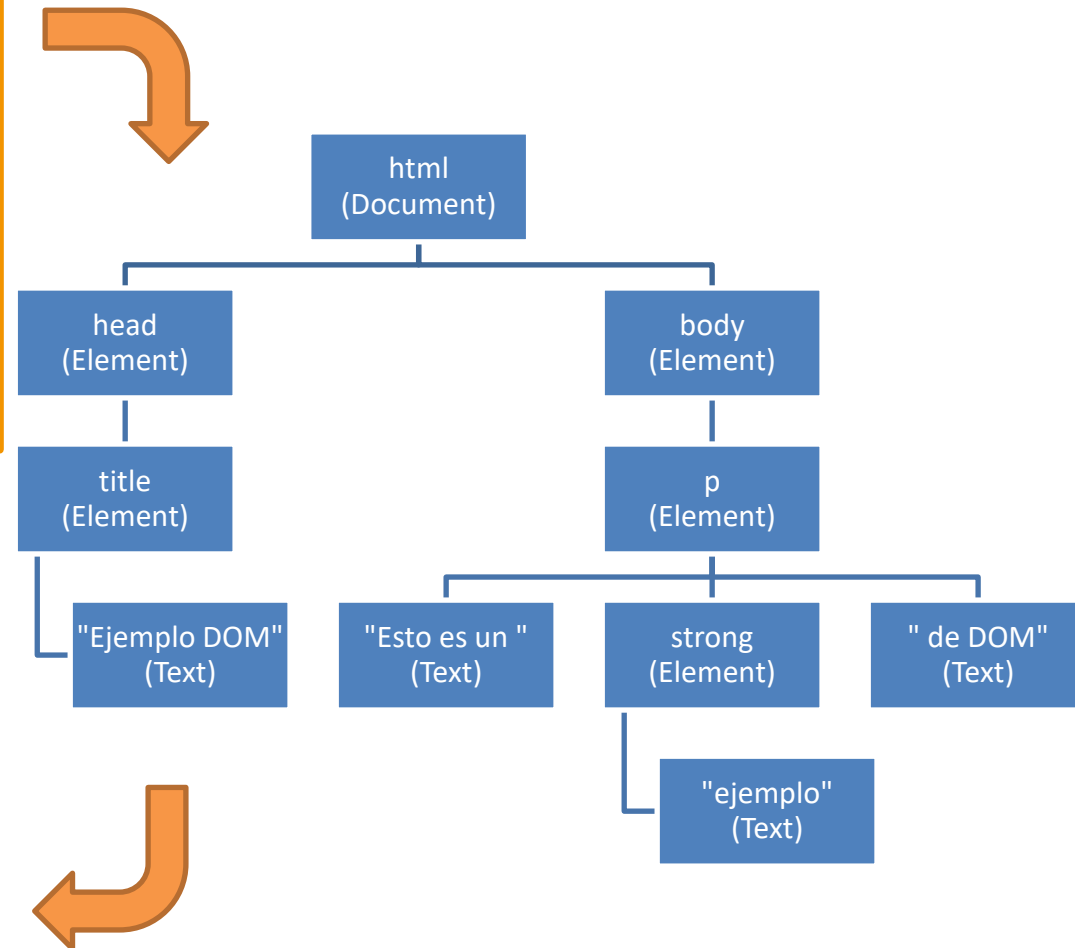
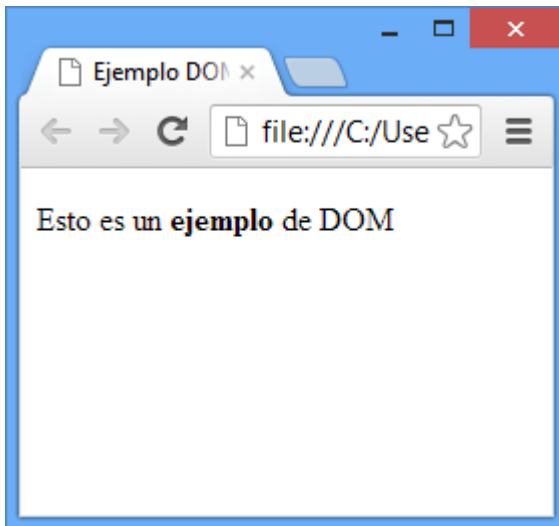
- **Document Object Model** o **DOM** (Modelo de Objetos del Documento)
- DOM trata un document HTML como un objeto formado por nodos
- DOM es independiente del navegador
- Permite modificar dinámicamente un documento HTML
- La navegación en un documento DOM es similar a recorrer una estructura arbolada

DOM

```
<!DOCTYPE html>
<html>
<head>
<title>Ejemplo DOM</title>
</head>

<body>
<p>Esto es un <strong>ejemplo</strong>
de DOM</p>
</body>

</html>
```



DOM



Nodos

- Los nodos son cualquier elemento que se representa en el árbol del DOM
 - Documento (raíz)
 - Elemento (definido entre una etiqueta de apertura y otra de cierre)
 - Atributo (de un elemento)
 - Texto (contenido de nodos textuales)
 - Comentario (`<!-- ... -->`)

Objeto document

- El acceso al DOM en Javascript se realiza mediante el objeto global **document**
- **document** encapsula todo el DOM de una página web
- Contiene propiedades y métodos que permiten acceder y modificar la página web

Objeto Document

```
<!DOCTYPE html>
<html>

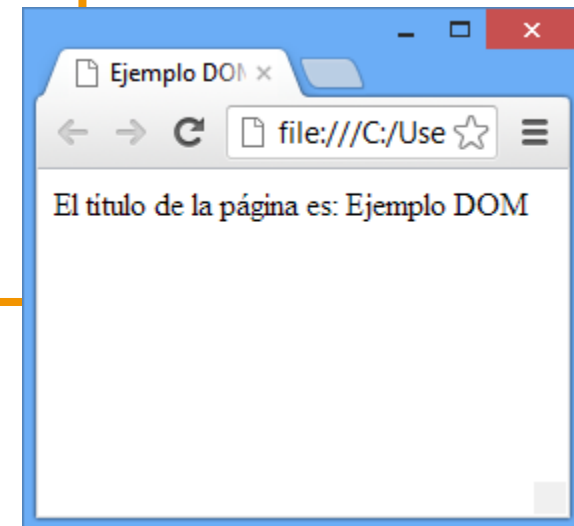
<head>
<title>Ejemplo DOM</title>
</head>

<body>

<script language="javascript">
document.write("El título de la página es: " + document.title);
</script>

</body>

</html>
```



Esquema de trabajo

- El esquema de trabajo normal con javascript en una pagina web es:

1. Obtener una referencia al objeto(s) con el que se quiere trabajar

```
// Obtengo una referencia al elemento con id 'uno'  
var elem = document.getElementById('uno');
```

2. Consultar o modificar propiedades o invocar métodos sobre ese objeto(s)

```
elem.style.color = "red"; // color rojo
```


Elementos

Obtiene una referencia al objeto (HTMLElement)

Modifica el contenido del elemento

```
<html>
<head>
<script type="text/javascript">
function cambiarEnlace()
{
  document.getElementById('miEnlace').innerHTML="Google";
  document.getElementById('miEnlace').href="http://google.com";
  document.getElementById('miEnlace').target="_blank";
}
</script>
</head>
<body>

<a id="miEnlace" href="http://www.microsoft.com">Microsoft</a>
<input type="button" onclick="cambiarEnlace()" value="Change link">

</body>
</html>
```

Modifica propiedades específicas del tipo de elemento

[Microsoft](#)

Después de pulsar el botón

[Google](#)

Referencias a elementos

- Para obtener una referencia a un elemento del DOM se pueden utilizar diversos métodos del objeto `document`

Método	Descripción
<code>getElementById(id)</code>	Obtiene una referencia del elemento con un id determinado
<code>getElementsByClassName(clase)</code>	Obtiene un array de elementos que tienen esa clase
<code>getElementsByTagName(etiqueta)</code>	Obtiene un array de elementos con ese nombre de etiqueta (div, p, a, ul, ol,...)
<code>querySelector(selectorCSS)</code>	Obtiene el primer elemento que cumple con el selector CSS indicado
<code>querySelectorAll(selectorCSS)</code>	Obtiene un array con todos los elementos que cumplen el selector CSS indicado

Recorridos DOM

- Una vez se tiene una referencia a un elemento se puede recorrer el árbol del DOM desde ese elemento.

```
var elem = document.getElementById('uno');  
var padre = elem.parentNode;
```

- Con estas propiedades se obtienen otras referencias a elementos relacionados

Propiedad	Descripción
parentNode	Elemento padre
firstChild	Primer nodo hijo (o null)
lastChild	Último hijo
childNodes	Array con los hijos del elemento

Creación de elementos

- Se puede crear elementos mediante el método **createElement**(nombreEtiqueta)

```
// Crea un párrafo nuevo <p></p>  
var p = document.createElement("p");
```

Inserción de elementos

- Se puede insertar un elemento como hijo de otro elemento:
 - Como **último** hijo del padre

```
padre.appendChild(nuevoHijo)
```

- **Antes** de otro elemento hijo del padre

```
padre.insertBefore(nuevoHijo, hijoReferencia)
```

Eliminar elementos

- Se puede eliminar un elemento mediante el método **remove()**

Atributos

- Se pueden obtener o modificar los atributos de un elemento

```
<input id="uno" type="text" />
```

```
Var elm = document.getElementById("uno");
```

Propiedad/Método	Descripción	Ejemplo
attributes	Todos los atributos de un elemento	<pre>var a = elm.attributes ; // array de referencias a atributos</pre>
hasAttribute (name)	Indica si un elemento tiene ese atributo	<pre>elm.hasAttribute("type"); // true elm.hasAttribute("max"); // false</pre>
getAttribute (name)	Obtiene el valor de un atributo	<pre>elm.getAttribute("type"); // "text"</pre>
setAttribute (name, value)	Establece el valor de un atributo	<pre>elm.setAttribute("type", "number"); // cambia type a "number"</pre>
removeAttribute (name)	Borra un atributo	<pre>elm.removeAttribute("type"); // Borra el atributo type</pre>

Propiedades especiales

Propiedad	Descripción	Ejemplo
tagName	Nombre de la etiqueta	"div"
id	Id del elemento	"uno"
className	Clase del elemento	"activo"
innerHTML	Contenido del elemento	"Hola Mundo"
value	Valor actual de un input (si el usuario ha <u>modificado</u> el valor del input, no se corresponde con el atributo)	
checked	Indica si un input de tipo checkbox está marcado	

```
<div id="uno" class="activo">Hola Mundo</div>
```


Gestión de clases

- Para gestionar las clases de un elemento se puede modificar el atributo class:
 - Con el método **setAttribute**("class", value)
 - Con la propiedad **className**
 - Mediante métodos específicos de **classList** que permiten gestionar la lista de clases de forma individual

```
<div id="uno" class="activo principal"></div>
```

Método	Descripción	Ejemplo
<code>classList.add</code> (clase)	Añade una clase	<code>elm.classList.add("ejemplo");</code>
<code>classList.remove</code> (clase)	Elimina una clase	<code>elm.classList.remove("activo");</code>
<code>classList.toggle</code> (clase)	Alterna una clase	<code>elm.classList.toggle("principal");</code> <code>elm.classList.toggle("secundario");</code>



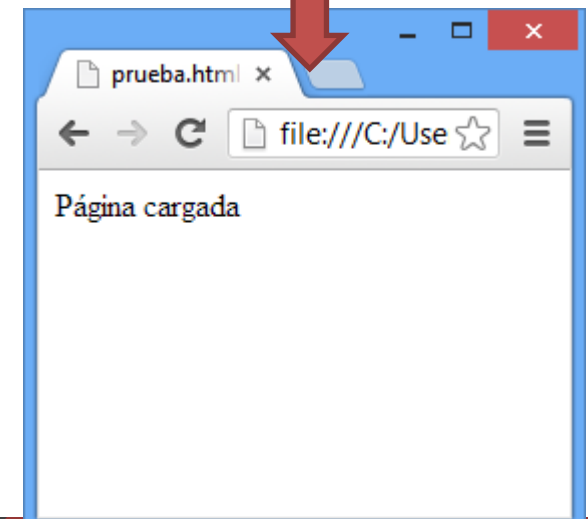
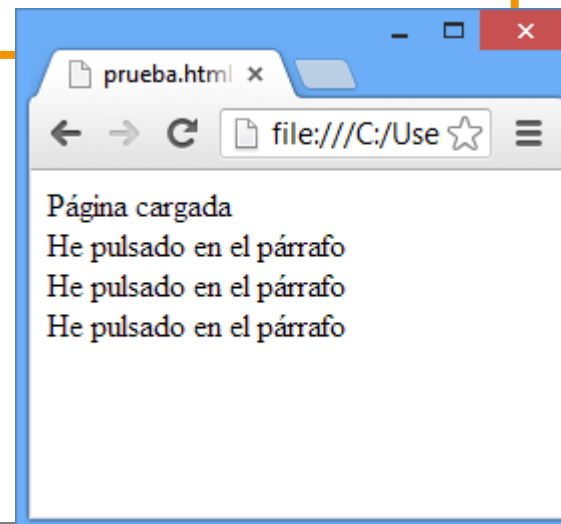
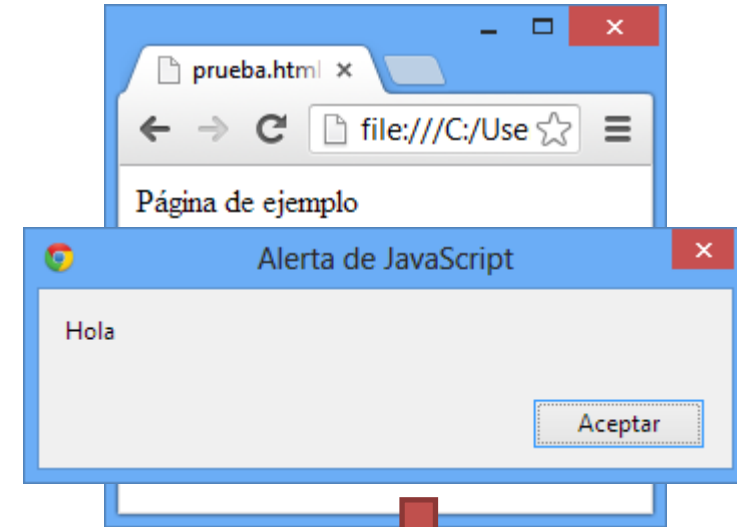
Eventos

Eventos

- Los **eventos** permiten ejecutar fragmentos de Javascript cuando el usuario interacciona con la web
 - Pulsaciones
 - Movimientos del ratón
 - Teclas
 - Carga de la página
 - ...

Eventos

```
<html>
<head>
<script type="text/javascript">
function inicio()
{
    alert("Hola");
    document.getElementById("ejemplo").innerHTML="Página cargada";
}
function pulsar()
{
    var texto = document.getElementById("ejemplo").innerHTML;
    texto += "<br>He pulsado en el párrafo";
    document.getElementById("ejemplo").innerHTML = texto;
}
</script>
</head>
<body onLoad="inicio()">
<p id="ejemplo" onClick="pulsar()">Página de ejemplo</p>
</body>
</html>
```



Eventos de ratón

Evento	Descripción
mouseover	El ratón entra en un elemento
mousemove	El ratón se mueve sobre un elemento
mouseout	El ratón sale de un elemento
mousedown	El botón ratón baja (al pulsar) sobre un elemento
mouseup	El botón del ratón sube (al soltar) sobre un elemento
click	Se hace un click sobre un elemento
dblclick	Se hace doble click sobre un elemento

Eventos de teclado

Evento	Descripción
keydown	Una tecla ha bajado al pulsar
keypress	Una tecla ha sido pulsada
keyup	Una tecla es soltada al pulsar

Eventos de interacción

Evento	Descripción	Dónde
focus	Un elemento obtiene el foco	
blur	Un elemento pierde el foco	
change	Un campo cambia su contenido	<input>
load	Una página o imagen se ha cargado	<body>,
unload	El usuario cierra la ventana	<body>

Eventos

```
<html>
<body>
Campo 1: <input type="text" id="campo1" value="Hola Mundo" />
<br />
Campo 2: <input type="text" id="campo2" />
<br />
<br />
Pulsa el botón para copiar el contenido
<br />
<button
onclick="document.getElementById('campo2').value=document.getElementById('campo1').value">
Copiar texto</button>
</body>
</html>
```

Campo 1:

Campo 2:

Pulsa el botón para copiar el contenido

Después de
pulsar el botón



Campo 1:

Campo 2:

Pulsa el botón para copiar el contenido

Manejadores

- Para indicar que código (función manejadora) se ejecuta cuando se produce un evento:
 - En el HTML indicar mediante un atributo **on***event* (onclick, onfocus, onload,...) el código a ejecutar

```
<button onclick="calcImpuestos()">Calcular Impuestos</button>
```

- Desde JavaScript utilizar el método **addEventListener**

```
<button id="calcImp">Calcular Impuestos</button>
```

```
var elm = document.getElementById("calcImp");  
elm.addEventListener("click", calcImpuestos);
```

Javascript – DOM



Infraestructura de
Sistemas Clínicos